

Softwarequalität

Hausarbeit

Studiengang Informatik

Duale Hochschule Baden-Württemberg Karlsruhe

Etienne Luke Josef Bader

Eingereicht am: 19.05.2026

Matrikelnummer, Kurs: 9578543, TINF23B2

Betreuer an der DHBW: Dennis Kube, Jonathan Schwarzenböck

Inhalt

1	Einleitung	1
1.1	Projektbeschreibung	1
1.2	Ziel der Arbeit	1
2	Grundlagen	3
2.1	Clean Architecture	3
2.1.1	Motivation und Ziel	3
2.1.2	Die Dependency Rule	3
2.1.3	Schichtenaufbau	4
2.1.4	Innere Schichten definieren Interfaces	4
2.2	Softwarequalität nach ISO/IEC 25010	5
3	Betrachtung des Projekts	6
3.1	Architekturaufbau des Projekts	6
3.2	Korrekte Umsetzung im Überblick	7
3.3	Strukturelle Abweichungen	7
3.3.1	Direkter Repository-Zugriff in der Plugin-Schicht	7
3.3.2	Framework-Abhängigkeit in der Application-Schicht	8
4	Analyse und Bewertung	9
4.1	Bewertung: Direkter Repository-Zugriff	9
4.1.1	Fehlende Kapselung der Anwendungslogik	9
4.1.2	Auswirkung auf Modularity	9
4.1.3	Auswirkung auf Modifiability	10
4.2	Bewertung: Framework-Abhängigkeit in der Application-Schicht	10
4.3	Risikobewertung mittels FMEA	10
4.4	Gesamtbewertung	12
5	Optimierungsmaßnahmen	13
5.1	Maßnahme 1: UseCase-Interfaces für alle Controller-Endpunkte (F1)	13
5.1.1	Beschreibung	13
5.1.2	Umsetzung im PDCA	14
5.1.3	Erwartete Auswirkung	14
5.2	Maßnahme 2: Spring-Abhängigkeit aus der Application-Schicht entfernen (F2). . .	
	14	
5.2.1	Beschreibung	14
5.2.2	Umsetzung im PDCA	14

5.2.3 Erwartete Auswirkung	15
5.3 Zusammenfassung der erwarteten Verbesserungen	15
6 Fazit	16
A Literatur	17
B Glossar	18



Einleitung

1.1 Projektbeschreibung

Das Sportwettbewerbs-Managementtool ist eine im Rahmen der Vorlesung Advanced Software Engineering an der Dualen Hochschule Baden-Württemberg Karlsruhe entwickelte Anwendung. Sie unterstützt Organisatorinnen und Organisatoren bei der Planung und Durchführung von Sportwettbewerben und bietet Funktionen zur Verwaltung von Personen, Teams, Wettkämpfen, Spielplänen, Lageplänen und Tickets. Das System besteht aus einem Java-Backend auf Basis des Spring-Boot-Frameworks sowie einem React-Frontend und wird über eine REST-API angesteuert.

Ein zentrales Merkmal des Projekts ist die bewusste Entscheidung für eine Clean Architecture. Diese gliedert das System in vier klar voneinander getrennte Schichten: Domain, Application, Adapter und Plugins. Ziel dieser Architektur ist es, die fachliche Kernlogik von technischen Details wie Frameworks oder Persistenzmechanismen zu entkoppeln und langfristige Wartbarkeit und Erweiterbarkeit sicherzustellen.

1.2 Ziel der Arbeit

Im Mittelpunkt dieser Hausarbeit steht die Frage, inwieweit die gewählte Architektur im Sinne des Softwarequalitätsmanagements tatsächlich konsequent umgesetzt wurde. Als Bewertungsmaßstab dienen zwei Teilmerkmale des Qualitätsmerkmals Maintainability aus dem internationalen Standard ISO/IEC 25010: Modularity und Modifiability. Modularity beschreibt, in welchem Maße ein System aus unabhängigen Komponenten besteht, sodass Änderungen an einer Komponente möglichst geringe Auswirkungen auf andere haben. Modifiability

beschreibt, wie effektiv das System verändert werden kann, ohne dabei unbeabsichtigte Seiteneffekte in anderen Teilen zu erzeugen.

Beide Teilmerkmale sind unmittelbar mit dem Anspruch der Clean Architecture verknüpft und eignen sich daher als konkreter, messbarer Maßstab zur Bewertung ihrer Umsetzung. Die Analyse konzentriert sich auf den `CompetitionController` als repräsentatives Fallbeispiel, an dem sich eine systematische Abweichung vom eigenen Architekturanspruch belegen lässt. Daraus werden anschließend begründete Optimierungsmaßnahmen abgeleitet.



Grundlagen

2.1 Clean Architecture

2.1.1 Motivation und Ziel

Moderne Softwaresysteme unterliegen einem beständigen Wandel. Technologien, Frameworks und Abhängigkeiten veralten, werden ersetzt oder weiterentwickelt – häufig in einem Rhythmus von wenigen Jahren [1, S. 2]. Eine Architektur, die eng an konkrete Technologien geknüpft ist, altert zwangsläufig mit diesen zusammen und erschwert spätere Anpassungen erheblich.

Die Clean Architecture begegnet diesem Problem durch eine klare strukturelle Trennung von langlebigem und kurzlebigem Code. Ihr zentrales Ziel ist es, einen technologieunabhängigen Kern zu schaffen und alle technischen Details – Datenbanken, Frameworks, Benutzeroberflächen – als austauschbare Randkomponenten zu behandeln [1, S. 9]. Das angestrebte Ergebnis ist ein System, in dem Technologieentscheidungen spät getroffen oder nachträglich revidiert werden können, ohne den Anwendungskern zu berühren [1, S. 8]. Robert C. Martin fasst dies so zusammen: Eine gute Architektur maximiert die Anzahl der Entscheidungen, die noch nicht getroffen werden müssen [2, S. 141].

2.1.2 Die Dependency Rule

Das zentrale Prinzip der Clean Architecture ist die Dependency Rule: Abhängigkeiten zwischen Systemteilen dürfen ausschließlich von außen nach innen zeigen [1, S. 11]. Dabei sind Abhängigkeitspfeile (Compile-Time Dependencies), die zeigen welchen Code eine Klasse

direkt referenziert, von Aufrufpfeilen (Runtime Dependencies) zu unterscheiden — letztere können in beide Richtungen verlaufen [1, S. 12].

Die Dependency Rule betrifft ausschließlich die Abhängigkeitspfeile: Innere Schichten dürfen äußere Schichten weder kennen noch referenzieren. Eine Verletzung dieser Regel untergräbt die gesamte Architektur, da Änderungen an der äußeren Schicht dann unmittelbar Auswirkungen auf den eigentlich stabilen Kern haben [2, S. 203].

2.1.3 Schichtenaufbau

Die Clean Architecture gliedert ein System typischerweise in vier Schichten [1, S. 14]:

Die **Domain-Schicht** bildet den innersten Kern mit den zentralen Geschäftsobjekten und organisationsweit gültigen Geschäftsregeln [1, S. 18–19]. Sie ist vollständig immun gegen Änderungen an Infrastrukturdetails wie Anzeige, Transport oder Speicherung [1, S. 18].

Die **Application-Schicht** enthält die anwendungsspezifischen Anwendungsfälle (Use Cases) und implementiert Regeln, die nur für den konkreten Anwendungsfall gelten [1, S. 23–24]. Sie ist isoliert von Änderungen an Datenbank oder Benutzeroberfläche [1, S. 25].

Die **Adapter-Schicht** vermittelt zwischen Anwendungslogik und Außenwelt durch Formatkonvertierungen [1, S. 30]. Ihr Ziel ist die vollständige Entkopplung von innen und außen — kein SQL in der Anwendung selbst, keine Renderlogik im Kern [1, S. 31].

Die **Plugin-Schicht** ist die äußerste Schicht mit Frameworks, Datenbank, Web und Benutzeroberfläche [1, S. 37]. Sie soll ausschließlich Delegationscode enthalten und darf keine Anwendungslogik enthalten — alle Entscheidungen sollen bereits in den inneren Schichten gefallen sein [1, S. 37–38]. Während Domain-Code Jahrzehnte Bestand haben kann, veralten Plugin-Code und Frameworks mitunter innerhalb von Wochen bis Monaten [1, S. 50].

2.1.4 Innere Schichten definieren Interfaces

Ein wesentliches Umsetzungsmittel der Dependency Rule ist die konsequente Nutzung von Interfaces: Innere Schichten definieren Schnittstellen, äußere Schichten implementieren diese [1, S. 16]. So kann die Domain-Schicht ein Repository-Interface definieren, ohne zu wissen, ob die konkrete Implementierung eine Datei, eine relationale Datenbank oder einen Webservice verwendet. Dieses als Dependency Inversion Principle bekannte Muster ist eine der tragenden Säulen der Clean Architecture [2, S. 91].

2.2 Softwarequalität nach ISO/IEC 25010

ISO/IEC 25010 ist der internationale Standard zur Beschreibung und Bewertung von Softwarequalität. Er definiert ein hierarchisches Qualitätsmodell, das Qualitätsmerkmale in Hauptmerkmale und Teilmerkmale untergliedert [3, Abschn. 4.2]. Als Bewertungsmaßstab für diese Arbeit sind zwei Teilmerkmale des Hauptmerkmals **Maintainability** (Wartbarkeit) relevant.

Modularity beschreibt den Grad, zu dem ein System aus voneinander unabhängigen Komponenten besteht, sodass eine Änderung an einer Komponente möglichst geringe Auswirkungen auf andere hat [3, Abschn. 4.2.7.1].

Modifiability beschreibt den Grad, zu dem ein System effektiv und effizient verändert werden kann, ohne dabei unbeabsichtigte Seiteneffekte in anderen Teilen zu erzeugen [3, Abschn. 4.2.7.3]. Ein System mit schlechter Modularität erschwert gezielte Modifikationen, da Änderungen unkontrolliert ausstrahlen.

Beide Teilmerkmale sind unmittelbar mit dem Anspruch der Clean Architecture verknüpft: Die Dependency Rule und die daraus resultierende Schichtentrennung sind präzise darauf ausgelegt, Modularity und Modifiability zu maximieren. Die Clean Architecture liefert damit nicht nur ein Designprinzip, sondern zugleich den Maßstab, an dem ihre eigene Umsetzung gemessen werden kann.

3

Betrachtung des Projekts

Dieses Kapitel beschreibt den Gesamtaufbau des Sportwettbewerbs-Managementtools in Bezug auf seine Architektur und arbeitet sowohl gelungene Aspekte als auch strukturelle Abweichungen heraus, die als Grundlage für die Analyse in Kapitel 4 dienen.

3.1 Architekturaufbau des Projekts

Das Backend des Projekts ist als Maven-Multi-Modul-Projekt organisiert und folgt dem in Kapitel 2 beschriebenen Schichtenmodell. Die vier Module sind entsprechend ihrer Schicht benannt: `3_domain`, `2_application`, `1_adapter` und `0_plugins`. Diese Nummerierung spiegelt die Abhängigkeitsrichtung wider und entspricht der Empfehlung von Briem, Schichten als separate Projekte umzusetzen, sodass der Compiler unzulässige Abhängigkeiten verhindert [1, S. 44].

Die **Domain-Schicht** (`3_domain`) enthält die zentralen Geschäftsobjekte `Competition`, `Match`, `Standings`, `Team`, `Person` und ihre Subtypen, `Ticket` sowie `Siteplan`. Zu jedem Aggregat existiert ein Repository-Interface in der Domain-Schicht. Die Domain-Schicht enthält keinerlei Framework-Imports, was der Grundregel der Clean Architecture entspricht [1, S. 16].

Die **Application-Schicht** (`2_application`) enthält für jeden fachlichen Bereich einen Service sowie die zugehörigen UseCase-Interfaces — etwa `CreateCompetitionUseCase` und `GetStandingsUseCase`. Alle UseCases kommunizieren ausschließlich über Command-Objekte und DTOs mit den äußeren Schichten.

Die **Adapter-Schicht** (1_adapter) enthält Mapper-Klassen, die zwischen Domain-Objekten und Persistenz-DTOs übersetzen. Diese Schicht hat keine Kenntnis von HTTP, REST oder Spring.

Die **Plugin-Schicht** (0_plugins) gliedert sich in plugin_rest mit den REST-Controllern und plugin_io mit den dateibasierten Repository-Implementierungen. Letztere implementieren jeweils das in der Domain-Schicht definierte Repository-Interface und erfüllen damit das Dependency-Inversion-Prinzip korrekt [1, S. 16].

3.2 Korrekte Umsetzung im Überblick

In weiten Teilen des Projekts ist die Clean Architecture konsequent umgesetzt. Der PersonController nutzt für alle vier Schreiboperationen ausschließlich UseCase-Interfaces ohne direkten Repository-Zugriff. Gleiches gilt für den TicketController, dessen beide schreibenden Endpunkte vollständig über CreateTicketUseCase und SellTicketUseCase abgewickelt werden [1, S. 37]. Auch die Domain-Schicht zeigt qualitativ hochwertige Umsetzungsbeispiele: Die Klasse SportResultComparator nutzt das Strategy-Pattern über das Interface ScoringStrategy, sodass neue Sportarten hinzugefügt werden können, ohne bestehenden Code zu verändern.

3.3 Strukturelle Abweichungen

Neben diesen gelungenen Aspekten finden sich im Projekt zwei strukturelle Abweichungen von der Clean Architecture, die für die Bewertung nach ISO/IEC 25010 relevant sind.

3.3.1 Direkter Repository-Zugriff in der Plugin-Schicht

Der CompetitionController hält als Instanzvariable nicht nur die UseCase-Interfaces CreateCompetitionUseCase und GetStandingsUseCase, sondern zusätzlich eine direkte Referenz auf CompetitionRepository. Von den sieben Endpunkten des Controllers umgehen fünf die Application-Schicht vollständig:

- GET /getById → competitionRepository.findCompetitionById(id)
- GET /getMatchesByCompetitionId → competitionRepository.getAllMatchesFromCompetitionId(id)
- GET /getMatchById → competitionRepository.findMatchById(id)
- GET /getCompetitionByCompetitionName → competitionRepository.getCompetitionByName(name)

- POST `/registerMatchResults`
→ `competitionRepository.registerResultsForMatchByID(...)`

Lediglich GET `/getStandingsFromCompetition` und POST `/create` nutzen den vorgesehenen Weg über UseCase-Interfaces. Die Plugin-Schicht greift damit für den Großteil ihrer Operationen direkt auf die Domain-Schicht zu – eine Verletzung der Dependency Rule [1, S. 37]. Das Muster findet sich auch im `PersonController` und `TicketController` (je ein Lesezugriff), ist jedoch im `CompetitionController` am ausgeprägtesten.

3.3.2 Framework-Abhängigkeit in der Application-Schicht

Die `pom.xml` der Application-Schicht deklariert `spring-boot-starter-web` als Compile-Zeit-Abhängigkeit. Dies hat zur Folge, dass alle fünf Service-Klassen die Spring-Annotation `@Service` tragen. Frameworks sind jedoch Details, die als Plugins an den Rand der Anwendung gehören – Abhängigkeiten vom Anwendungscode in das Framework zeigen in die falsche Richtung [1, S. 58–59]. Die Application-Schicht kann damit nicht mehr unabhängig von Spring kompiliert oder getestet werden, was einem der Grundziele der Clean Architecture widerspricht [1, S. 16].

4

Analyse und Bewertung

Dieses Kapitel bewertet die in Kapitel 3 beschriebenen Abweichungen anhand der eingeführten Kriterien und quantifiziert den Handlungsbedarf mittels FMEA.

4.1 Bewertung: Direkter Repository-Zugriff

4.1.1 Fehlende Kapselung der Anwendungslogik

Die Application-Schicht dient nicht allein als Durchleitungsebene, sondern als zentraler Ort für anwendungsspezifische Geschäftslogik [1, S. 23]. Dies zeigt sich deutlich am Vergleich mit den Endpunkten, die den vorgesehenen Weg nutzen: Der Aufruf von `createCompetitionUseCase.create(command)` löst in `CompetitionService` eine Kette fachlicher Schritte aus — Auflösung der Team- und Official-IDs, automatische Spielplanerstellung mittels `competition.scheduleMatches()` sowie Persistierung. Beim direkten Repository-Zugriff ist diese Logik entweder im Controller dupliziert oder schlicht nicht vorhanden. Das Plugin enthält damit Anwendungslogik — genau das, was Briem als grundlegenden Verstoß gegen die Rolle der Plugin-Schicht beschreibt [1, S. 37–38].

4.1.2 Auswirkung auf Modularity

Im vorliegenden Fall hängt der `CompetitionController` durch den direkten Import von `CompetitionRepository` und mehreren Domain-Klassen (`Competition`, `Match`, `Standings`) unmittelbar von der Domain-Schicht ab. Plugin- und Domain-Schicht sind damit direkt

aneinander gekoppelt, obwohl die Application-Schicht als Entkopplungsebene vorgesehen ist. Eine Änderung an der Signatur einer Repository-Methode würde damit nicht nur die Application-Schicht betreffen, sondern unmittelbar auch den Controller — ein direkter Verstoß gegen das Ziel der Modularity [3, Abschn. 4.2.7.1]. Im Vergleich dazu hätte eine Änderung an `PersonRepository` keine direkte Auswirkung auf den `PersonController`, da dieser das Repository nicht kennt.

4.1.3 Auswirkung auf Modifiability

Soll künftig beim Abrufen eines Wettkampfs eine Berechtigungsprüfung oder Logging-Funktion eingebaut werden, müsste dies direkt im Controller implementiert werden. In einem System mit konsequenter Clean Architecture wäre dieser Eingriff auf die Application-Schicht beschränkt; der Controller bliebe unverändert. Zusätzlich besteht eine Inkonsistenz innerhalb des `CompetitionController` selbst: Zwei Endpunkte laufen korrekt über UseCase-Interfaces, fünf nicht. Diese Inkonsistenz erhöht die kognitive Last bei Weiterentwicklungen erheblich und widerspricht dem Grundsatz, dass die Plugin-Schicht ausschließlich Delegationscode enthalten soll [1, S. 37].

4.2 Bewertung: Framework-Abhängigkeit in der Application-Schicht

Die Application-Schicht deklariert `spring-boot-starter-web` in ihrer `pom.xml` als Compile-Zeit-Abhängigkeit. Damit kennt und benötigt die Application-Schicht Spring, um kompiliert werden zu können — eine strukturelle Verletzung der Dependency Rule, wonach Abhängigkeiten von außen nach innen zeigen sollen [1, S. 11]. Die praktische Konsequenz ist, dass die Application-Schicht nicht mehr unabhängig von Spring gebaut oder betrieben werden kann [1, S. 16]. Hinsichtlich Modularity würde ein Wechsel des Web-Frameworks nun auch die Application-Schicht berühren, obwohl er laut Clean Architecture ausschließlich die Plugin-Schicht betreffen sollte. Hinsichtlich Modifiability erfordern Komponententests der Services Spring-Kontext oder Spring-Mocking-Infrastruktur, obwohl die fachliche Logik von Spring vollständig unabhängig sein sollte [3, Abschn. 4.2.7.3].

4.3 Risikobewertung mittels FMEA

Um die identifizierten Befunde strukturiert zu priorisieren, wird eine Fehler-Möglichkeiten- und Einfluss-Analyse (FMEA) durchgeführt [4, S. 10]. Jede Fehlerquelle wird anhand dreier Faktoren bewertet:

- **A** – Auftrittswahrscheinlichkeit (1–10)
- **B** – Bedeutung der Fehlerfolge (1–10)
- **E** – Entdeckungswahrscheinlichkeit (1 = zwangsläufig entdeckt, 10 = kaum entdeckbar)

Die Risikoprioritätszahl ergibt sich als $RPZ = A \times B \times E$. Werte ab 100 gelten als hohes Risiko mit dringendem Handlungsbedarf [4, S. 10].

ID	Fehlerquelle	A	B	E	RPZ
F1	Direkter Repository-Zugriff im CompetitionController (5 von 7 Endpunkten umgehen die Application-Schicht)	10	7	8	560
F2	Spring-Compile-Abhängigkeit in der Application-Schicht (spring-boot-starter-web in pom.xml)	10	5	4	200

Tabelle 1 – FMEA-Analyse der identifizierten Architekturverletzungen

RPZ	Risiko	Handlungsbedarf	Maßnahmen
RPZ = 1	keins	keiner	keine
2–50	gering	nicht zwingend	können formuliert und umgesetzt werden
50–100	mittel	besteht	sollten formuliert und umgesetzt werden
100–1000	hoch	dringend	müssen formuliert und umgesetzt werden

Tabelle 2 – RPZ-Wertebereiche und Fehlerrisikoklassen (nach [4, S. 10])

Beide Fehlerquellen fallen in die Risikoklasse **hoch**. F1 erhält $A = 10$, da die Verletzung bereits vollständig im Code vorhanden ist. $B = 7$ spiegelt die substanzielle Beeinträchtigung beider Qualitätskriterien wider. $E = 8$, da die Verletzung im normalen Entwicklungsbetrieb kaum auffällt – der Code kompiliert fehlerfrei und funktionale Tests würden keine Abweichung zeigen. F2 erhält $A = 10$ und $B = 5$, da die funktionalen Auswirkungen begrenzter sind; $E = 4$, da die Abhängigkeit in der `pom.xml` explizit sichtbar ist.

4.4 Gesamtbewertung

Beide Befunde sind Ausdruck desselben Grundproblems: Die Dependency Rule wurde an der Grenze zwischen Plugin-Schicht und Application-Schicht nicht konsequent eingehalten. Im Kern – Domain- und Adapter-Schicht – ist die Architektur vorbildlich umgesetzt. An der äußersten Grenze entstehen jedoch Kurzschlussverbindungen, die Modularity und Modifiability untergraben. Die FMEA-Analyse bestätigt diesen Befund quantitativ und stuft beide Fehlerquellen als dringend ein [4, S. 10]. F1 ist dabei als dringlicher zu bewerten, da seine Auswirkungen breiter und schwerer zu kontrollieren sind.

5

Optimierungsmaßnahmen

Ausgehend von der FMEA werden konkrete Maßnahmen erarbeitet, geordnet nach RPZ. Die Umsetzung folgt dem PDCA-Kreislauf als iterativem Qualitätsverbesserungsprozess [4, S. 5].

5.1 Maßnahme 1: UseCase-Interfaces für alle Controller-Endpunkte (F1)

5.1.1 Beschreibung

Für jeden Endpunkt des CompetitionController, der derzeit direkt auf CompetitionRepository zugreift, wird ein eigenes UseCase-Interface eingeführt:

- GET /getById → GetCompetitionByIdUseCase
- GET /getMatchesByCompetitionId → GetMatchesByCompetitionUseCase
- GET /getMatchById → GetMatchByIdUseCase
- GET /getCompetitionByCompetitionName → GetCompetitionByNameUseCase
- POST /registerMatchResults → RegisterMatchResultsUseCase

Der CompetitionController wird so umgebaut, dass er ausschließlich UseCase-Interfaces als Abhängigkeiten hält. Alle Domain-Imports werden entfernt; der Controller kommuniziert nur noch über Command-Objekte und DTOs – analog zur bereits korrekten Umsetzung im PersonController.

5.1.2 Umsetzung im PDCA

Plan: Die neuen UseCase-Interfaces werden spezifiziert. Jedes Interface erhält genau eine Methode, die ein Command-Objekt entgegennimmt und ein DTO zurückgibt.

Do: Die Interfaces werden in der Application-Schicht angelegt, CompetitionService implementiert sie, und der CompetitionController wird umgestellt.

Check: Der CompetitionController darf keine import-Anweisungen aus `de.dhbw.ase.domain` mehr enthalten. Dies lässt sich automatisiert durch ArchUnit in der CI-Pipeline prüfen.

Act: Das gleiche Muster wird auf PersonController und TicketController übertragen, sodass alle Controller einheitlich nur über UseCase-Interfaces kommunizieren.

5.1.3 Erwartete Auswirkung

Plugin-Schicht und Domain-Schicht werden vollständig entkoppelt. Die Compile-Time-Abhängigkeiten des CompetitionController reduzieren sich von zwei Schichten auf eine. Fachliche Erweiterungen erhalten einen definierten Ort in der Application-Schicht, und alle sieben Endpunkte folgen demselben konsistenten Muster.

5.2 Maßnahme 2: Spring-Abhängigkeit aus der Application-Schicht entfernen (F2)

5.2.1 Beschreibung

`spring-boot-starter-web` wird aus der `pom.xml` der Application-Schicht entfernt. Die `@Service`-Annotierungen aller fünf Service-Klassen werden durch eine `@Configuration`-Klasse in der Plugin-Schicht ersetzt, die die Services explizit als `@Bean` registriert.

5.2.2 Umsetzung im PDCA

Plan: Eine neue Klasse `ApplicationConfig` wird in `plugin_rest` angelegt. Da die Services Abhängigkeiten bereits per Konstruktor-Injektion erhalten, sind keine weiteren Anpassungen an den Service-Klassen erforderlich.

Do: Die `@Service`-Annotierungen werden aus allen Services entfernt, `spring-boot-starter-web` wird aus der `pom.xml` gestrichen, und `ApplicationConfig` übernimmt die Bean-Registrierung.

Check: `mvn compile -pl 2_application` muss ohne Fehler durchlaufen. Bestehende Unit-Tests bleiben unverändert lauffähig, da sie mit Mockito arbeiten und keinen Spring-Kontext benötigen.

Act: Zukünftige Services werden von Beginn an ohne Framework-Annotierungen entwickelt.

5.2.3 Erwartete Auswirkung

Die Application-Schicht wird vollständig framework-agnostisch. Ein Wechsel des Web-Frameworks betrifft ausschließlich die Plugin-Schicht — dies entspricht dem Ziel der Clean Architecture, Technologien als austauschbare Randkomponenten zu behandeln [1, S. 52–53].

5.3 Zusammenfassung der erwarteten Verbesserungen

Nach Einführung von ArchUnit-Tests sinkt E für F1 von 8 auf 2; die RPZ reduziert sich von 560 auf 140. Nach Entfernung der Spring-Abhängigkeit sinkt B für F2 von 5 auf 2; die RPZ reduziert sich von 200 auf 80, was der Risikoklasse „mittel“, entspricht. Beide Maßnahmen zusammen führen zu einem System, das die Grundregeln der Clean Architecture konsequent einhält: Innere Schichten definieren Interfaces, äußere implementieren diese, und jede Schicht ist unabhängig kompilierbar und testbar [1, S. 16].



Fazit

Diese Arbeit hat das Sportwettbewerbs-Managementtool anhand der Teilmerkmale Modularity und Modifiability des ISO/IEC-25010-Standards untersucht und dabei die Clean Architecture als konkreten Bewertungsmaßstab herangezogen.

Die Analyse zeigt ein zweigeteiltes Bild. Im Kern – der Domain- und Adapter-Schicht – ist die Clean Architecture vorbildlich umgesetzt. An der Schichtgrenze zwischen Plugin- und Application-Schicht finden sich zwei strukturelle Abweichungen: Der `CompetitionController` umgeht die Application-Schicht für fünf von sieben Endpunkten durch direkten Repository-Zugriff, und die Application-Schicht hält eine Compile-Zeit-Abhängigkeit auf Spring Boot. Beide Befunde wurden mittels FMEA als hohes Risiko eingestuft (RPZ 560 und 200) und beeinträchtigen nachweislich Modularity und Modifiability.

Die erarbeiteten Optimierungsmaßnahmen – Einführung fehlender UseCase-Interfaces und Verlagerung der Spring-Konfiguration in die Plugin-Schicht – sind gezielt und mit überschaubarem Aufwand umsetzbar. Durch automatisierte Architekturprüfung mittels ArchUnit schaffen sie zudem eine nachhaltige Absicherung gegen künftige Regressionen.

Das Fallbeispiel illustriert ein in der Praxis häufiges Muster: Eine Architekturentscheidung wird konzeptionell korrekt getroffen, in der Umsetzung jedoch unter Zeitdruck nicht konsequent durchgehalten. Genau hier setzt Softwarequalitätsmanagement an – nicht als nachträgliche Kritik, sondern als Instrument, um solche Abweichungen systematisch zu identifizieren, zu bewerten und dauerhaft zu beheben [4, S. 5].

A Literatur

- [1] L. Briem, „Clean Architecture: Systeme für die Ewigkeit“, 2024.
- [2] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Pearson Education, 2018.
- [3] ISO/IEC, „Systems and software engineering — Systems and software quality requirements and evaluation (SQuaRE) — System and software quality models“, Nr. ISO/IEC25010:2011. 2011.
- [4] D. Kube und J. Schwarzenböck, „Softwarequalität“, 2023.

B Glossar

Selbstständigkeitserklärung

Gemäß Ziffer 1.1.13 der Anlage 1 zu §§ 3, 4 und 5 der Studien- und Prüfungsordnung für die Bachelorstudiengänge im Studienbereich Technik der Dualen Hochschule Baden- Württemberg vom 29.09.2017. Ich versichere hiermit, dass ich meine Arbeit mit dem Thema:

Softwarequalität

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass alle eingereichten Fassungen übereinstimmen.

Karlsruhe, 19.05.2026

Etienne Luke Josef Bader